

TÀI LIỆU HƯỚNG DẪN THỰC HÀNH

TRIỂN KHAI APACHE CASSANDRA TRÊN DATASTAX ASTRA DB

Bài toán demo: Hệ thống Quản lý Khách sạn (Hotel Management)

Môi trường thực hành – Lưu trữ dữ liệu – Project khai thác dữ liệu NoSQL

MỤC LỤC

- 1. Mục tiêu và đối tượng sử dụng
- 2. Tổng quan Apache Cassandra & DataStax Astra DB
- 3. Đăng ký tài khoản và khởi tạo Database trên Astra DB
- 4. Làm quen với CQL Console
- 5. Bài toán demo: Hệ thống Quản lý Khách sạn
- 6. Thiết kế dữ liệu theo tư duy Query-First của Cassandra
- 7. Khởi tạo Keyspace và các bảng dữ liệu (CQL Script)
- 8. Nạp dữ liệu mẫu
- 9. Khai thác dữ liệu: các truy vấn CQL thực hành
- 10. Kết nối ứng dụng ngoài bằng Driver (Python)
- 11. Bài tập và gợi ý đề tài Project khai thác dữ liệu
- 12. Một số lưu ý và lỗi thường gặp
- 13. Tài liệu tham khảo

1. MỤC TIÊU VÀ ĐỐI TƯỢNG SỬ DỤNG

1.1. Mục tiêu

Sau khi hoàn thành tài liệu hướng dẫn này, sinh viên có khả năng:

- Hiểu được đặc điểm của cơ sở dữ liệu NoSQL dạng cột mở rộng (Wide-column) mà đại diện là Apache Cassandra, phân biệt với mô hình quan hệ (RDBMS).
- Tự đăng ký và khởi tạo một môi trường Cassandra thật (miễn phí) trên nền tảng đám mây DataStax Astra DB, không cần cài đặt phức tạp trên máy cá nhân.
- Thực hành thiết kế dữ liệu theo tư duy "Query-First" (thiết kế bảng dựa trên truy vấn) đặc trưng của Cassandra, thông qua bài toán Quản lý khách sạn.
- Viết được các câu lệnh CQL (Cassandra Query Language) để tạo Keyspace, tạo bảng, nạp dữ liệu và truy vấn khai thác dữ liệu.
- Kết nối được từ ứng dụng bên ngoài (Python) tới Astra DB bằng driver chính thức để xây dựng một project khai thác dữ liệu hoàn chỉnh.

1.2. Đối tượng sử dụng

Tài liệu dành cho sinh viên đã học qua kiến thức cơ bản về cơ sở dữ liệu (SQL) và có kiến thức lập trình nền tảng (Python hoặc Java/Node.js), đang thực hành nội dung NoSQL / Cassandra trong chương trình đào tạo.

Lưu ý về công cụ được đề xuất

Toàn bộ hướng dẫn trong tài liệu này sử dụng DataStax Astra DB — dịch vụ Managed Cassandra trên Cloud do chính đội ngũ đóng góp chính cho Apache Cassandra phát triển. Sinh viên KHÔNG cần cài đặt Cassandra trên máy cá nhân, không cần cấu hình cluster, chỉ cần trình duyệt web và kết nối Internet.

2. TỔNG QUAN APACHE CASSANDRA & DATASTAX ASTRA DB

2.1. Apache Cassandra là gì?

Apache Cassandra là hệ quản trị cơ sở dữ liệu NoSQL mã nguồn mở, thuộc nhóm Wide-column Store, được thiết kế để lưu trữ và xử lý khối lượng dữ liệu rất lớn, phân tán trên nhiều máy chủ (node) mà không có điểm lỗi trung tâm (No Single Point of Failure). Cassandra được sử dụng bởi nhiều hệ thống lớn như Netflix, Apple, Instagram nhờ khả năng:

- Khả năng mở rộng theo chiều ngang (Horizontal Scalability): thêm node để tăng dung lượng và hiệu năng, không cần nâng cấp phần cứng của một máy chủ duy nhất.
- Tính sẵn sàng cao (High Availability): dữ liệu được sao chép (replication) trên nhiều node, hệ thống vẫn hoạt động khi một số node gặp sự cố.
- Không có kiến trúc Master-Slave: mọi node đều bình đẳng (Peer-to-Peer), tránh điểm nghẽn và điểm lỗi trung tâm.
- Mô hình dữ liệu linh hoạt dựa trên Keyspace → Table → Partition Key → Clustering Column, tối ưu cho việc đọc/ghi theo truy vấn đã biết trước.

2.2. So sánh nhanh với cơ sở dữ liệu quan hệ (RDBMS)

Tiêu chí	RDBMS (MySQL, SQL Server...)	Apache Cassandra
Mô hình dữ liệu	Bảng quan hệ, chuẩn hoá (normalization)	Wide-column, phi chuẩn hoá (denormalization) theo truy vấn
Thiết kế	Thiết kế trước, viết truy vấn sau (Data-first)	Xác định truy vấn trước, thiết kế bảng sau (Query-first)
JOIN giữa các bảng	Hỗ trợ tốt	Không hỗ trợ — dữ liệu được nhân bản (duplicate) theo truy vấn
Khả năng mở rộng	Chủ yếu mở rộng theo chiều dọc (Vertical)	Mở rộng theo chiều ngang (Horizontal) rất mạnh
Tính nhất quán	ACID chặt chẽ	Tunable Consistency (có thể tùy chỉnh mức độ nhất quán)

2.3. DataStax Astra DB — vì sao nên dùng để thực hành?

Website: astra.datastax.com

DataStax là công ty đóng góp chính (major contributor) cho dự án mã nguồn mở Apache Cassandra. Astra DB là dịch vụ Managed Cassandra trên nền Cloud do chính DataStax vận hành, giúp người dùng có một cụm Cassandra thật sự chạy trên Cloud chỉ trong vài phút, không cần tự cài đặt và quản trị hạ tầng.

Lý do tài liệu này khuyến nghị sử dụng Astra DB cho môi trường thực hành của sinh viên:

- Có gói Free Tier vĩnh viễn: miễn phí trong hạn mức sử dụng hằng tháng, không bắt buộc phải khai báo thẻ thanh toán ngay lúc đăng ký để học tập, thực hành.

- Cung cấp sẵn CQL Console ngay trên trình duyệt, tương đương với việc gõ lệnh cqlsh thật trên máy — không cần cài đặt Cassandra hay cqlsh cục bộ.
- Hỗ trợ tạo Keyspace, thiết kế bảng dữ liệu trực quan và kết nối từ bên ngoài bằng driver chính thức cho Java, Python, Node.js một cách thuận tiện.
- Là một dịch vụ Cloud thật, giúp sinh viên làm quen với quy trình làm việc thực tế (đăng ký tài khoản Cloud, quản lý Token/Secure Connect Bundle, bảo mật kết nối) — kỹ năng cần thiết khi đi làm.

3. ĐĂNG KÝ TÀI KHOẢN VÀ KHỞI TẠO DATABASE TRÊN ASTRA DB

3.1. Đăng ký tài khoản

Bước 1: Truy cập trang đăng ký — mở trình duyệt và vào địa chỉ astra.datastax.com.

Bước 2: Chọn phương thức đăng ký — có thể đăng ký nhanh bằng tài khoản Google, GitHub, hoặc bằng email cá nhân/email trường (khuyến khích dùng email trường để thuận tiện quản lý cho lớp học).

Bước 3: Xác thực email — nếu đăng ký bằng email, hệ thống sẽ gửi một email xác thực — sinh viên vào hộp thư và nhấn vào liên kết xác nhận để kích hoạt tài khoản.

Bước 4: Hoàn tất hồ sơ — đăng nhập vào Astra Portal (trang quản trị), điền một số thông tin cơ bản nếu được yêu cầu (tên tổ chức có thể để mặc định).

Không cần thẻ thanh toán

Với gói Free Tier, sinh viên KHÔNG bắt buộc phải khai báo thẻ tín dụng để bắt đầu sử dụng cho mục đích học tập.

3.2. Tạo Database (cụm Cassandra) mới

Bước 1: Vào mục Databases — trong Astra Portal, chọn mục "Databases" ở thanh điều hướng bên trái, sau đó chọn nút Create Database.

Bước 2: Chọn loại Database — chọn loại "Serverless" (hoặc "Classic — Non-Vector" tùy giao diện hiện tại), phù hợp cho việc thực hành CQL/Cassandra thuần túy (không cần tính năng Vector Search cho AI).

Bước 3: Đặt tên Database — ví dụ: `hotel_management_demo`. Lưu ý: tên Database KHÔNG thể đổi lại sau khi tạo.

Bước 4: Đặt tên Keyspace ban đầu — ví dụ: `hotel_ks`. Keyspace tương đương khái niệm "Schema/Database" trong SQL — là không gian chứa các bảng dữ liệu liên quan.

Bước 5: Chọn nhà cung cấp Cloud và khu vực (Region) — chọn Google Cloud/AWS/Azure và khu vực gần nhất (ví dụ Singapore/Asia) để giảm độ trễ khi kết nối.

Bước 6: Xác nhận khởi tạo — nhấn Create Database. Quá trình khởi tạo cụm mất khoảng 2–5 phút, trạng thái sẽ chuyển từ Pending/Initializing sang Active. Hệ thống gửi email thông báo khi Database sẵn sàng.

3.3. Lấy thông tin kết nối và bảo mật

Sau khi Database ở trạng thái Active, vào tab của Database vừa tạo để lấy các thông tin quan trọng cho việc kết nối sau này:

- **Application Token:** chuỗi mã xác thực (dạng "AstraCS:...") dùng để kết nối ứng dụng/driver tới Database — có vai trò tương tự username/password. Tạo mới tại mục "Application Tokens" và LƯU LẠI ngay vì hệ thống chỉ hiển thị một lần.
- **Secure Connect Bundle (SCB):** tệp .zip chứa chứng chỉ bảo mật (SSL) và thông tin cụm, dùng khi kết nối bằng driver Java/Python theo phương thức Cloud Config truyền thống.

- API Endpoint: đường dẫn REST/Data API của Database (nếu sử dụng phương thức kết nối mới qua Data API thay vì driver CQL truyền thống).

Bảo mật Token

KHÔNG chia sẻ Application Token hoặc đưa Token trực tiếp vào code khi nộp bài/đẩy code lên GitHub công khai. Nên lưu Token trong biến môi trường (environment variable) hoặc tệp cấu hình được loại trừ khỏi Git (.gitignore).

4. LÀM QUEN VỚI CQL CONSOLE

CQL Console là giao diện dòng lệnh (tương đương cqlsh) được Astra DB tích hợp sẵn ngay trên trình duyệt, giúp sinh viên gõ và chạy lệnh CQL (Cassandra Query Language) mà không cần cài đặt bất kỳ phần mềm nào.

4.1. Truy cập CQL Console

Bước 1: Mở Database — từ Astra Portal, nhấn vào tên Database đã tạo (hotel_management_demo).

Bước 2: Chọn tab CQL Console — trên thanh tab của trang quản trị Database, chọn "CQL Console".

Bước 3: Chờ kết nối — hệ thống tự động kết nối và hiển thị dấu nhắc lệnh dạng token@cqlsh>, sẵn sàng nhận lệnh CQL.

4.2. Một số lệnh CQL kiểm tra nhanh

```
-- Xem danh sách các keyspace hiện có
DESCRIBE KEYSPACES;

-- Chuyển sang làm việc trong keyspace của môn học
USE hotel_ks;

-- Xem danh sách bảng trong keyspace hiện tại
DESCRIBE TABLES;
```

Từ đây, sinh viên có thể gõ trực tiếp các lệnh CQL ở Phần 7 và Phần 8 vào CQL Console để tạo bảng và nạp dữ liệu cho bài toán demo.

5. BÀI TOÁN DEMO: HỆ THỐNG QUẢN LÝ KHÁCH SẠN

Đề sinh viên có một môi trường thực hành gắn liền với bài toán thực tế, tài liệu sử dụng nghiệp vụ Quản lý khách sạn (Hotel Management) làm ví dụ xuyên suốt cho việc thiết kế dữ liệu, tạo bảng, nạp dữ liệu và khai thác dữ liệu bằng CQL.

5.1. Mô tả nghiệp vụ

- Một chuỗi khách sạn (hotel chain) quản lý nhiều khách sạn (hotel) đặt tại nhiều thành phố khác nhau.
- Mỗi khách sạn có nhiều phòng (room) với các loại phòng khác nhau (Standard, Deluxe, Suite) và mức giá khác nhau.
- Khách hàng (guest) có thể đặt phòng (booking) tại một khách sạn trong một khoảng thời gian (ngày nhận phòng – ngày trả phòng).
- Mỗi lượt đặt phòng phát sinh một hoá đơn (invoice) ghi nhận chi phí và trạng thái thanh toán.
- Khách sạn cần tra cứu nhanh: phòng còn trống theo ngày, lịch sử đặt phòng của một khách hàng, danh sách đặt phòng của một khách sạn theo khoảng thời gian, doanh thu theo khách sạn/theo ngày.

5.2. Xác định các truy vấn nghiệp vụ cần hỗ trợ

Đây là bước quan trọng nhất khi làm việc với Cassandra: phải liệt kê trước TẤT CẢ các truy vấn (query) mà hệ thống cần phục vụ, vì cấu trúc bảng sẽ được thiết kế xoay quanh các truy vấn này (chi tiết ở Phần 6).

Mã	Truy vấn nghiệp vụ cần hỗ trợ
Q1	Xem thông tin chi tiết của một khách sạn theo mã khách sạn (hotel_id).
Q2	Xem danh sách phòng và trạng thái của một khách sạn.
Q3	Xem toàn bộ lượt đặt phòng của một khách hàng, sắp xếp theo ngày nhận phòng gần nhất.
Q4	Xem danh sách đặt phòng của một khách sạn trong một khoảng ngày (phục vụ báo cáo, kiểm tra phòng trống).
Q5	Tra cứu thông tin hoá đơn theo mã đặt phòng (booking_id).

6. THIẾT KẾ DỮ LIỆU THEO TƯ DUY QUERY-FIRST

Khác với SQL (thiết kế bảng chuẩn hoá trước, viết truy vấn linh hoạt sau), Cassandra yêu cầu tư duy ngược lại: mỗi truy vấn quan trọng ở Phần 5.2 nên có một bảng được thiết kế riêng, tối ưu cho đúng truy vấn đó. Việc dữ liệu bị lặp lại (denormalization) giữa các bảng là bình thường và được chấp nhận, đổi lại tốc độ đọc dữ liệu cực nhanh.

6.1. Nguyên tắc chọn Partition Key và Clustering Column

- Partition Key: quyết định dữ liệu được lưu trên node vật lý nào — nên chọn cột thường xuất hiện trong mệnh đề WHERE để tra cứu (ví dụ `hotel_id`, `guest_id`).
- Clustering Column: quyết định thứ tự sắp xếp dữ liệu bên trong một partition — dùng cho các cột cần lọc theo khoảng hoặc cần sắp xếp (ví dụ `check_in_date`).
- Một bảng Cassandra thường được đặt tên theo mẫu `<thực_thể>_by_<tiêu_chí_truy_vấn>` để thể hiện rõ mục đích phục vụ truy vấn nào, ví dụ `bookings_by_guest`, `bookings_by_hotel_date`.

6.2. Ánh xạ truy vấn → bảng dữ liệu

Truy vấn	Bảng thiết kế	Partition Key	Clustering Column
Q1	hotels	hotel_id	—
Q2	rooms_by_hotel	hotel_id	room_number
Q3	bookings_by_guest	guest_id	check_in_date, booking_id (giảm dần)
Q4	bookings_by_hotel_date	hotel_id	check_in_date, booking_id
Q5	invoices_by_booking	booking_id	—

Vì sao dữ liệu "đặt phòng" (booking) được lưu ở 2 bảng khác nhau?

Vì Cassandra không hỗ trợ JOIN, nếu hệ thống cần tra cứu đặt phòng theo cả hai chiều (theo khách hàng — Q3, và theo khách sạn/ngày — Q4), cách làm đúng là tạo hai bảng riêng biệt (`bookings_by_guest` và `bookings_by_hotel_date`), cùng lưu các cột dữ liệu cần thiết, rồi ghi dữ liệu vào cả hai bảng cùng lúc khi có một lượt đặt phòng mới. Đây chính là kỹ thuật denormalization đặc trưng của Cassandra.

7. KHỞI TẠO KEYSPACE VÀ CÁC BẢNG DỮ LIỆU

Keyspace `hotel_ks` đã được tạo sẵn cùng lúc với Database ở Bước 3.2. Sinh viên gõ trực tiếp các lệnh dưới đây vào CQL Console (sau khi `USE hotel_ks;`) để tạo các bảng phục vụ 5 truy vấn đã xác định.

7.1. Bảng `hotels` — thông tin khách sạn (phục vụ Q1)

```
CREATE TABLE IF NOT EXISTS hotels (
    hotel_id      TEXT,
    hotel_name    TEXT,
    city          TEXT,
    address       TEXT,
    star_rating   INT,
    phone         TEXT,
    PRIMARY KEY (hotel_id)
);
```

7.2. Bảng `rooms_by_hotel` — danh sách phòng theo khách sạn (Q2)

```
CREATE TABLE IF NOT EXISTS rooms_by_hotel (
    hotel_id      TEXT,
    room_number   INT,
    room_type     TEXT,
    price_per_night DECIMAL,
    status        TEXT,      -- 'AVAILABLE' | 'OCCUPIED' | 'MAINTENANCE'
    PRIMARY KEY (hotel_id, room_number)
);
```

7.3. Bảng `bookings_by_guest` — lịch sử đặt phòng theo khách hàng (Q3)

```
CREATE TABLE IF NOT EXISTS bookings_by_guest (
    guest_id      TEXT,
    check_in_date DATE,
    booking_id    UUID,
    hotel_id      TEXT,
    hotel_name    TEXT,
    room_number   INT,
    check_out_date DATE,
    total_amount  DECIMAL,
    status        TEXT,      -- 'CONFIRMED' | 'CHECKED_IN' | 'CHECKED_OUT' | 'CANCELLED'
    PRIMARY KEY (guest_id, check_in_date, booking_id)
) WITH CLUSTERING ORDER BY (check_in_date DESC, booking_id DESC);
```

7.4. Bảng `bookings_by_hotel_date` — đặt phòng theo khách sạn/ngày (Q4)

```
CREATE TABLE IF NOT EXISTS bookings_by_hotel_date (
    hotel_id      TEXT,
    check_in_date DATE,
    booking_id    UUID,
    guest_id      TEXT,
```

```
    guest_name    TEXT,  
    room_number   INT,  
    check_out_date DATE,  
    total_amount  DECIMAL,  
    status        TEXT,  
    PRIMARY KEY ((hotel_id), check_in_date, booking_id)  
  ) WITH CLUSTERING ORDER BY (check_in_date ASC, booking_id ASC);
```

7.5. Bảng `invoices_by_booking` — hoá đơn theo lượt đặt phòng (Q5)

```
CREATE TABLE IF NOT EXISTS invoices_by_booking (  
    booking_id    UUID,  
    invoice_id    UUID,  
    guest_id      TEXT,  
    hotel_id      TEXT,  
    room_charge   DECIMAL,  
    service_charge DECIMAL,  
    tax           DECIMAL,  
    total_amount  DECIMAL,  
    payment_status TEXT,    -- 'UNPAID' | 'PAID' | 'REFUNDED'  
    issued_at     TIMESTAMP,  
    PRIMARY KEY (booking_id, invoice_id)  
);
```

Kiểm tra lại toàn bộ bảng vừa tạo:

```
DESCRIBE TABLES;
```

8. NẠP DỮ LIỆU MẪU

Sinh viên chạy lần lượt các lệnh INSERT dưới đây trong CQL Console để có dữ liệu mẫu phục vụ thực hành truy vấn ở Phần 9. Lưu ý: vì bookings_by_guest và bookings_by_hotel_date cùng biểu diễn một lượt đặt phòng, cần dùng CHUNG một giá trị booking_id (UUID) khi INSERT vào cả hai bảng.

8.1. Dữ liệu khách sạn và phòng

```
INSERT INTO hotels (hotel_id, hotel_name, city, address, star_rating, phone)
VALUES ('HTL001', 'Saigon Riverside Hotel', 'Ho Chi Minh City', '123 Ton Duc Thang, Q1', 4, '028-1234567');

INSERT INTO hotels (hotel_id, hotel_name, city, address, star_rating, phone)
VALUES ('HTL002', 'Hanoi Old Quarter Hotel', 'Ha Noi', '45 Hang Bac, Hoan Kiem', 3, '024-7654321');

INSERT INTO rooms_by_hotel (hotel_id, room_number, room_type, price_per_night, status)
VALUES ('HTL001', 101, 'Standard', 800000, 'AVAILABLE');

INSERT INTO rooms_by_hotel (hotel_id, room_number, room_type, price_per_night, status)
VALUES ('HTL001', 102, 'Deluxe', 1200000, 'OCCUPIED');

INSERT INTO rooms_by_hotel (hotel_id, room_number, room_type, price_per_night, status)
VALUES ('HTL001', 201, 'Suite', 2500000, 'AVAILABLE');
```

8.2. Dữ liệu đặt phòng (ghi đồng thời vào 2 bảng)

```
-- Tạo trước 1 UUID dùng chung cho cả 2 bảng, ví dụ:
-- booking_id = 3f2504e0-4f89-11d3-9a0c-0305e82c3301

INSERT INTO bookings_by_guest
  (guest_id, check_in_date, booking_id, hotel_id, hotel_name, room_number,
   check_out_date, total_amount, status)
VALUES
  ('GUEST001', '2026-10-01', 3f2504e0-4f89-11d3-9a0c-0305e82c3301,
   'HTL001', 'Saigon Riverside Hotel', 102, '2026-10-03', 2400000, 'CONFIRMED');

INSERT INTO bookings_by_hotel_date
  (hotel_id, check_in_date, booking_id, guest_id, guest_name, room_number,
   check_out_date, total_amount, status)
VALUES
  ('HTL001', '2026-10-01', 3f2504e0-4f89-11d3-9a0c-0305e82c3301,
   'GUEST001', 'Nguyen Van A', 102, '2026-10-03', 2400000, 'CONFIRMED');
```

8.3. Dữ liệu hoá đơn

```
INSERT INTO invoices_by_booking
  (booking_id, invoice_id, guest_id, hotel_id, room_charge, service_charge,
   tax, total_amount, payment_status, issued_at)
VALUES
```

```
(3f2504e0-4f89-11d3-9a0c-0305e82c3301, uuid(), 'GUEST001', 'HTL001',  
2400000, 100000, 250000, 2750000, 'UNPAID', toTimestamp(now()));
```

Gợi ý thực hành

Sinh viên nên viết thêm ít nhất 10–15 dòng dữ liệu mẫu cho mỗi bảng (nhiều khách sạn, nhiều khách hàng, nhiều ngày đặt phòng khác nhau) để các truy vấn ở Phần 9 và bài tập ở Phần 11 có ý nghĩa thống kê rõ ràng hơn.

9. KHAI THÁC DỮ LIỆU: CÁC TRUY VẤN CQL THỰC HÀNH

Phần này minh hoạ cách các bảng đã thiết kế ở Phần 6–7 phục vụ trực tiếp cho từng truy vấn nghiệp vụ, đúng tinh thần Query-First của Cassandra.

9.1. Q1 — Thông tin chi tiết một khách sạn

```
SELECT * FROM hotels WHERE hotel_id = 'HTL001';
```

9.2. Q2 — Danh sách phòng và trạng thái của một khách sạn

```
SELECT room_number, room_type, price_per_night, status
FROM rooms_by_hotel
WHERE hotel_id = 'HTL001';

-- Lọc thêm phòng đang trống (cần thêm room_number vào điều kiện
-- hoặc lọc phía client, vì status không phải Clustering Column):
SELECT room_number, room_type, price_per_night
FROM rooms_by_hotel
WHERE hotel_id = 'HTL001' AND status = 'AVAILABLE' ALLOW FILTERING;
```

9.3. Q3 — Lịch sử đặt phòng của một khách hàng (mới nhất trước)

```
SELECT check_in_date, hotel_name, room_number, check_out_date, total_amount, status
FROM bookings_by_guest
WHERE guest_id = 'GUEST001';
```

9.4. Q4 — Đặt phòng của một khách sạn trong khoảng ngày (kiểm tra phòng trống)

```
SELECT check_in_date, room_number, guest_name, check_out_date, status
FROM bookings_by_hotel_date
WHERE hotel_id = 'HTL001'
AND check_in_date >= '2026-10-01' AND check_in_date <= '2026-10-31';
```

9.5. Q5 — Tra cứu hoá đơn theo mã đặt phòng

```
SELECT * FROM invoices_by_booking
WHERE booking_id = 3f2504e0-4f89-11d3-9a0c-0305e82c3301;
```

9.6. Một số hàm thống kê nhanh trên CQL

```
-- Đếm số phòng của một khách sạn
SELECT COUNT(*) FROM rooms_by_hotel WHERE hotel_id = 'HTL001';

-- Giá phòng cao nhất/thấp nhất trong một khách sạn (trên tập kết quả trả về)
SELECT MAX(price_per_night), MIN(price_per_night)
FROM rooms_by_hotel WHERE hotel_id = 'HTL001';
```

Vì sao không nên lạm dụng ALLOW FILTERING?

ALLOW FILTERING cho phép CQL lọc trên các cột không phải Partition/Clustering Key, nhưng Cassandra phải quét toàn bộ partition để lọc, làm mất đi lợi thế tốc độ vốn có. Trong thực tế, nếu một truy vấn lọc thường xuyên theo cột nào đó (ví dụ status), nên thiết kế thêm một bảng mới (ví dụ rooms_by_hotel_status) cho đúng tư duy Query-First, thay vì dùng ALLOW FILTERING lâu dài.

10.1. KẾT NỐI ỨNG DỤNG NGOÀI BẰNG DRIVER (PYTHON)

Sau khi đã làm chủ CQL trên Console, sinh viên tiến hành xây dựng một chương trình Python nhỏ để đọc/ghi dữ liệu vào Astra DB — đây là bước chuyển từ "thao tác thủ công" sang "ứng dụng thật", chuẩn bị cho project khai thác dữ liệu ở Phần 11.

10.1. Cài đặt thư viện

```
pip install cassandra-driver python-dotenv
```

10.2. Chuẩn bị thông tin kết nối

Tạo tệp .env (không đưa lên Git) chứa các biến sau, lấy từ Astra Portal theo hướng dẫn ở Mục 3.3:

```
ASTRA_DB_APPLICATION_TOKEN=AstraCS:xxxxxxxxxxxxxxxxxxxxxxxxxxxxx
ASTRA_DB_SECURE_BUNDLE_PATH=./secure-connect-hotel-management-demo.zip
ASTRA_DB_KEYSPACE=hotel_ks
```

10.3. Chương trình kết nối và truy vấn mẫu

```
import os
from dotenv import load_dotenv
from cassandra.cluster import Cluster
from cassandra.auth import PlainTextAuthProvider

load_dotenv()

cloud_config = {
    'secure_connect_bundle': os.getenv('ASTRA_DB_SECURE_BUNDLE_PATH')
}
auth_provider = PlainTextAuthProvider(
    'token', os.getenv('ASTRA_DB_APPLICATION_TOKEN')
)
cluster = Cluster(cloud=cloud_config, auth_provider=auth_provider)
session = cluster.connect(os.getenv('ASTRA_DB_KEYSPACE'))

# Q3: Lịch sử đặt phòng của một khách hàng
rows = session.execute(
    "SELECT check_in_date, hotel_name, room_number, total_amount, status "
    "FROM bookings_by_guest WHERE guest_id = %s",
    ('GUEST001',)
)
for row in rows:
    print(row.check_in_date, row.hotel_name, row.room_number,
          row.total_amount, row.status)

cluster.shutdown()
```

Ghi dữ liệu vào nhiều bảng bằng ứng dụng

Khi tạo một lượt đặt phòng mới từ ứng dụng, chương trình cần tự sinh một UUID dùng chung (ví dụ bằng `uuid.uuid4()` trong Python) rồi thực hiện hai lệnh INSERT — một vào `bookings_by_guest` và một vào `bookings_by_hotel_date` — để đảm bảo dữ liệu nhất quán giữa hai bảng, đúng như thao tác thủ công ở Mục 8.2.

10.2. KẾT NỐI ỨNG DỤNG NGOÀI BẰNG DRIVER (JAVA)

Bên cạnh Python, DataStax cung cấp driver chính thức cho Java — ngôn ngữ phổ biến trong các hệ thống doanh nghiệp. Phần này hướng dẫn xây dựng một chương trình Java kết nối tới Astra DB bằng DataStax Java Driver 4.x (CqlSession), sử dụng cùng bài toán Quản lý khách sạn ở Phần 5–9.

10.1. Khai báo thư viện (Maven)

Thêm dependency sau vào tệp pom.xml của project (Maven):

```
<dependency>
  <groupId>com.datastax.oss</groupId>
  <artifactId>java-driver-core</artifactId>
  <version>4.17.0</version>
</dependency>
```

Nếu dùng Gradle, khai báo tương ứng:

```
implementation 'com.datastax.oss:java-driver-core:4.17.0'
```

10.2. Chuẩn bị thông tin kết nối

Tải Secure Connect Bundle (tệp .zip) của Database từ Astra Portal theo hướng dẫn ở Mục 3.3, đặt vào thư mục src/main/resources của project (ví dụ đặt tên secure-connect-hotel-management-demo.zip). Tạo Application Token riêng cho ứng dụng Java (khuyến khích tạo Token mới, khác với Token dùng cho Python ở Phần 10 cũ, để dễ theo dõi/thu hồi khi cần).

Không hard-code Token trong mã nguồn

Nên truyền Token và đường dẫn Secure Connect Bundle qua biến môi trường (System.getenv(...)) hoặc tệp cấu hình application.conf nằm ngoài phạm vi commit Git, tương tự khuyến nghị đã nêu ở Mục 10.2 phần Python.

10.3. Chương trình kết nối và truy vấn mẫu

Ví dụ dưới đây khởi tạo CqlSession, thực hiện truy vấn Q3 (lich sử đặt phòng của một khách hàng) và ghi một lượt đặt phòng mới vào đồng thời hai bảng bookings_by_guest và bookings_by_hotel_date bằng BatchStatement để đảm bảo tính nhất quán:

```
import com.datastax.oss.driver.api.core.CqlSession;
import com.datastax.oss.driver.api.core.cql.*;
import java.nio.file.Paths;
import java.time.LocalDate;
import java.util.UUID;

public class HotelBookingDemo {

    public static void main(String[] args) {
        String token = System.getenv("ASTRA_DB_APPLICATION_TOKEN");
        String bundlePath = System.getenv("ASTRA_DB_SECURE_BUNDLE_PATH");
```

```

String keyspace = System.getenv("ASTRA_DB_KEYSPACE"); // hotel_ks

try (CqlSession session = CqlSession.builder()
    .withCloudSecureConnectBundle(Paths.get(bundlePath))
    .withAuthCredentials("token", token)
    .withKeyspace(keyspace)
    .build()) {

    // ---- Q3: Lịch sử đặt phòng của một khách hàng ----
    PreparedStatement psSelect = session.prepare(
        "SELECT check_in_date, hotel_name, room_number, total_amount, status " +
        "FROM bookings_by_guest WHERE guest_id = ?");
    ResultSet rs = session.execute(psSelect.bind("GUEST001"));
    for (Row row : rs) {
        System.out.printf("%s | %s | Phòng %d | %s | %s%n",
            row.getLocalDate("check_in_date"),
            row.getString("hotel_name"),
            row.getInt("room_number"),
            row.getBigDecimal("total_amount"),
            row.getString("status"));
    }

    // ---- Ghi một lượt đặt phòng mới vào 2 bảng bằng BATCH ----
    UUID bookingId = UUID.randomUUID();

    PreparedStatement psByGuest = session.prepare(
        "INSERT INTO bookings_by_guest (guest_id, check_in_date, booking_id, " +
        "hotel_id, hotel_name, room_number, check_out_date, total_amount, status) "
+
        "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)");

    PreparedStatement psByHotelDate = session.prepare(
        "INSERT INTO bookings_by_hotel_date (hotel_id, check_in_date, booking_id, "
+
        "guest_id, guest_name, room_number, check_out_date, total_amount, status) "
+
        "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)");

    BatchStatement batch = BatchStatement.builder(BatchType.LOGGED)
        .addStatement(psByGuest.bind(
            "GUEST002", LocalDate.of(2026, 11, 5), bookingId,
            "HTL001", "Saigon Riverside Hotel", 201,
            LocalDate.of(2026, 11, 7), new java.math.BigDecimal("5000000"),
"CONFIRMED"))
        .addStatement(psByHotelDate.bind(
            "HTL001", LocalDate.of(2026, 11, 5), bookingId,
            "GUEST002", "Tran Thi B", 201,
            LocalDate.of(2026, 11, 7), new java.math.BigDecimal("5000000"),
"CONFIRMED"))
        .build();

    session.execute(batch);
    System.out.println("Da tao dat phong moi: " + bookingId);
}
}

```

```
}
```

Vì sao dùng `BatchStatement` ở đây?

`BatchStatement` (loại `LOGGED`) giúp Cassandra đảm bảo cả hai lệnh `INSERT` vào `bookings_by_guest` và `bookings_by_hotel_date` cùng thành công hoặc cùng thất bại, tránh tình trạng dữ liệu lệch giữa hai bảng như đã cảnh báo ở Mục 12 (bảng lỗi thường gặp). Lưu ý: không nên lạm dụng `BATCH` cho số lượng lớn câu lệnh không liên quan tới nhau, vì có thể làm giảm hiệu năng ghi.

11. BÀI TẬP VÀ GỢI Ý ĐỀ TÀI PROJECT KHAI THÁC DỮ LIỆU

11.1. Bài tập thực hành cơ bản (làm trực tiếp trên CQL Console)

- Thiết kế và tạo thêm bảng guests (guest_id, full_name, phone, email, national_id) để lưu hồ sơ khách hàng.
- Viết câu lệnh UPDATE để cập nhật trạng thái phòng (status) khi khách nhận phòng (check-in) và khi trả phòng (check-out).
- Viết câu lệnh DELETE để huỷ một lượt đặt phòng (cần xoá đồng thời ở cả bookings_by_guest và bookings_by_hotel_date).
- Thiết kế thêm bảng revenue_by_hotel_month để phục vụ truy vấn doanh thu theo khách sạn theo từng tháng — xác định rõ Partition Key và Clustering Column phù hợp.

11.2. Gợi ý đề tài Project khai thác dữ liệu (làm bằng ứng dụng Python/Java/Node.js)

Đề tài	Mô tả yêu cầu chính
Hệ thống đặt phòng khách sạn trực tuyến (mini)	Xây dựng ứng dụng console hoặc web đơn giản cho phép tra cứu phòng trống, tạo đặt phòng mới, huỷ đặt phòng, kết nối trực tiếp tới Astra DB.
Dashboard thống kê doanh thu khách sạn	Đọc dữ liệu từ bookings_by_hotel_date và invoices_by_booking, tổng hợp và trực quan hoá doanh thu theo khách sạn/theo tháng bằng thư viện biểu đồ (ví dụ matplotlib, Chart.js).
Hệ thống cảnh báo phòng trống theo thời gian thực	Xây dựng script định kỳ quét bảng rooms_by_hotel, gửi cảnh báo (email/log) khi tỉ lệ phòng trống của một khách sạn vượt ngưỡng.
Phân tích hành vi đặt phòng của khách hàng	Khai thác bookings_by_guest để thống kê: khách sạn được đặt nhiều nhất, mùa cao điểm, khách hàng thân thiết (đặt phòng nhiều lần).

11.3. Tiêu chí gợi ý khi đánh giá project

- Tính đúng đắn của thiết kế dữ liệu: các bảng có được thiết kế theo tư duy Query-First, có giải thích được lý do chọn Partition Key/Clustering Column hay không.
- Tính nhất quán dữ liệu khi ghi vào nhiều bảng liên quan (ví dụ bookings_by_guest và bookings_by_hotel_date).
- Khả năng vận hành thực tế: chương trình kết nối, đọc/ghi dữ liệu thành công tới Astra DB thật (không phải dữ liệu giả lập cục bộ).
- Chất lượng trình bày, giải thích được sự khác biệt trong tư duy thiết kế giữa Cassandra và SQL truyền thống khi thuyết trình.

12. MỘT SỐ LƯU Ý VÀ LỖI THƯỜNG GẶP

Tình huống / Lỗi	Nguyên nhân thường gặp	Cách xử lý
InvalidRequest: Cannot execute this query as it might involve data filtering...	Truy vấn WHERE trên cột không phải Partition/Clustering Key.	Thêm ALLOW FILTERING (chỉ nên dùng tạm thời khi thực hành) hoặc thiết kế lại bảng theo đúng truy vấn (khuyến nghị).
NoHostAvailable / kết nối thất bại từ Python	Sai đường dẫn Secure Connect Bundle, Token hết hạn/bị thu hồi, hoặc mạng chặn cổng kết nối.	Kiểm tra lại đường dẫn file .zip, tạo lại Application Token mới, kiểm tra kết nối Internet/Firewall.
Database ở trạng thái Hibernated (đã "ngủ")	Gói Free Tier tự động tạm ngưng (hibernate) Database sau một thời gian không có hoạt động để tiết kiệm tài nguyên.	Vào Astra Portal, nhấn nút "Resume"/"Wake up" trên Database; chờ khoảng 1–2 phút để Database hoạt động trở lại.
Dữ liệu ở hai bảng liên quan bị lệch nhau (một bảng có, bảng kia không)	Chỉ INSERT/UPDATE vào một trong hai bảng biểu diễn cùng một thực thể (ví dụ chỉ sửa bookings_by_guest mà quên bookings_by_hotel_date).	Luôn thực hiện đồng thời các lệnh ghi liên quan; với ứng dụng thực tế nên dùng BATCH statement của CQL để đảm bảo các lệnh ghi liên quan được gửi cùng lúc.

13. TÀI LIỆU THAM KHẢO

- DataStax Astra DB — trang chủ và đăng ký: astra.datastax.com
- DataStax Docs — Astra DB Documentation: docs.datastax.com
- Apache Cassandra — trang chủ dự án mã nguồn mở: cassandra.apache.org
- Cassandra Query Language (CQL) Reference: cassandra.apache.org/doc/latest/cassandra/cql
- DataStax Python Driver for Apache Cassandra: docs.datastax.com/en/developer/python-driver